UNIVERSIDAD NACIONAL DE SAN AGUSTÍN DE AREQUIPA
Facultad de Ingeniería de Producción y Servicios
Escuela Profesional de Ingeniería de Sistemas

INFORME DE ARQUITECTURA

Agente Supervisor de Consentimientos (AGT-CONSENT-01)

Arquitectura lógica, técnica y de despliegue del agente basado en RAG híbrido con supervisión humana

Versión 1.0 – Segundo entregable

Curso:
Auditoría de Sistemas

Docente:
Zárate Carlos

Integrantes:
Bedregal Pérez, Daniel
Choquecondo Aspilcueta, Daniela
Payehuanca Riquelme, Jhastyn

Arequipa – Perú
2026

1. Introducción

1.1. Propósito del documento

Este informe describe la arquitectura del **Agente Supervisor de Consentimientos (AGT-CONSENT-01)**: sus componentes, cómo se relacionan, el flujo de ejecución, los datos que maneja, sus interfaces, su despliegue y los controles que garantizan resultados trazables. Complementa al Trabajo de Investigación Formativa (TIF), que expone la fundamentación normativa y los resultados; aquí el foco es **cómo está construido el agente** y por qué se tomaron sus decisiones de diseño.

1.2. Alcance del sistema

El agente evalúa evidencias documentales (formularios, avisos o políticas de privacidad y registros operativos) frente a los controles A.1.2.2, A.1.2.4, A.1.2.5, A.1.3.3, A.1.3.4 y A.1.3.5 de la ISO/IEC 27701:2025 y los Artículos 5, 13.5, 14 y 18 de la Ley N.° 29733. Para cada evidencia emite un dictamen por dominio (información y propósito, obtención y registro, modificación y revocación, y excepciones legales), con la norma aplicada, la cita textual de la evidencia y las medidas correctivas. Los casos dudosos no se emiten de forma autónoma: pasan a la revisión de un auditor humano.

Atributo	Valor
Identificador	AGT-CONSENT-01
Tipo de agente	Agente de auditoría con RAG híbrido, reglas deterministas, LLM local y supervisión humana (HITL)
Lenguaje y núcleo	Python 3.10+; el núcleo en modo local usa solo la biblioteca estándar
Repositorio	github.com/Dan-Ar5/Auditoria (rama main)
Calidad verificada	45 pruebas automatizadas aprobadas; validación reproducible de 5 casos

2. Vista de Contexto

La Figura 1 muestra el agente como una caja negra y los actores y sistemas con los que interactúa. El **equipo auditado** aporta la evidencia, que se carga manualmente como texto; el **auditor líder o DPO** revisa y decide sobre los informes retenidos; el **asistente OpenCode** puede invocar al agente mediante su skill y sus herramientas MCP. Como servicios de apoyo, el agente usa **Ollama** (modelo Llama 3 local) para razonar, **Qdrant** como índice vectorial y el **corpus normativo** como base de conocimiento. Con línea discontinua se indica la integración prevista con el agente de inventario del grupo (AGT-DATA-INV-01), que le derivará las actividades del RoPA cuya base legal es el consentimiento.

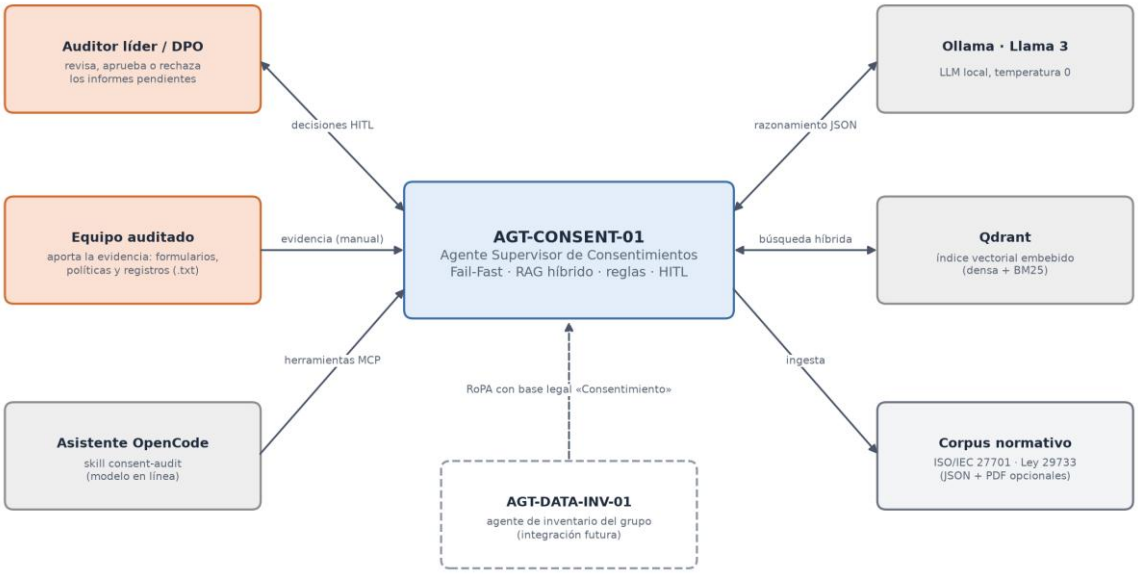


Figura 1: Diagrama de contexto del agente

3. Arquitectura Lógica por Capas

La solución se organiza en cuatro capas con responsabilidades separadas (Figura 2):

- **Capa de fuentes:** evidencia auditada, corpus normativo, PDF oficiales opcionales, reglas de auditoría, catálogo de excepciones del Art. 14 e índice de Qdrant.
- **Capa MCP:** un bus de herramientas con contratos explícitos (recuperar normas, verificar excepciones, analizar evidencia, emitir informe y auditoría completa) que desacopla el núcleo de las fuentes y del motor de recuperación.
- **Núcleo del agente:** orquesta el Fail-Fast, la recuperación híbrida por dominio, las reglas, el razonamiento del LLM y las guardas anti-alucinación.
- **Capa de gobernanza y salidas:** aplica las reglas de escalamiento HITL, mantiene la cola del auditor y la bitácora, y genera los entregables.

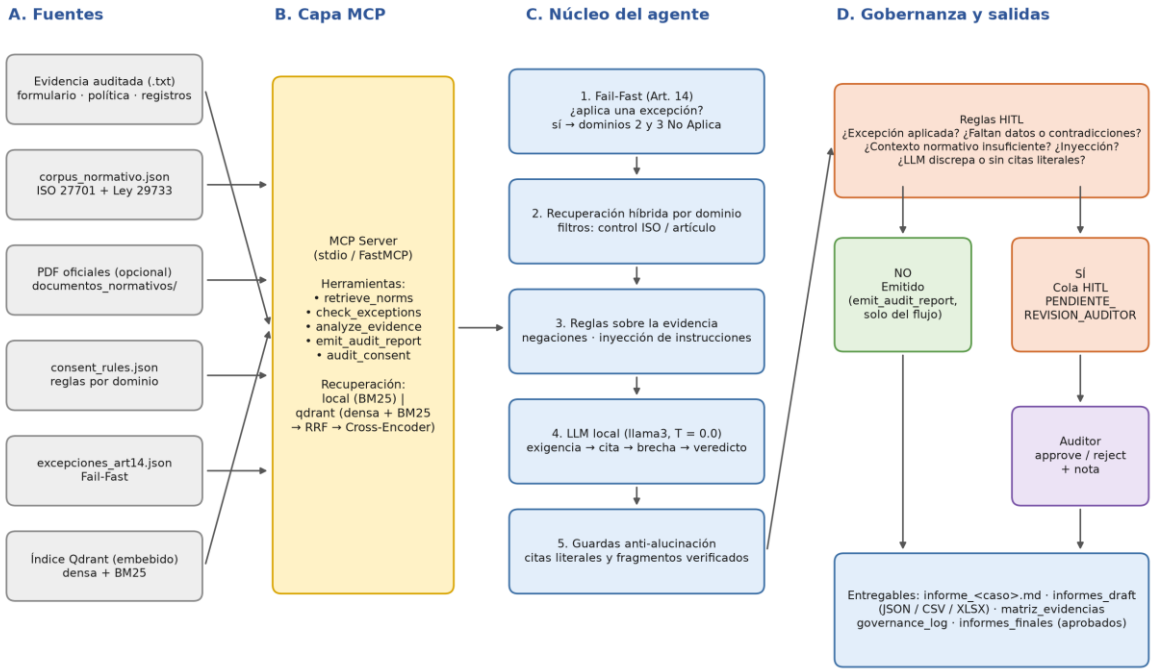


Figura 2: Arquitectura lógica por capas con punto de control humano

4. Vista de Componentes

La Figura 3 detalla los módulos del paquete `agente/` y los puntos de entrada que lo utilizan. Todos los puntos de entrada construyen el agente con la misma fábrica (`build_agent`), de modo que la consola, el panel web, el servidor MCP y la validación reproducible ejecutan exactamente la misma lógica.

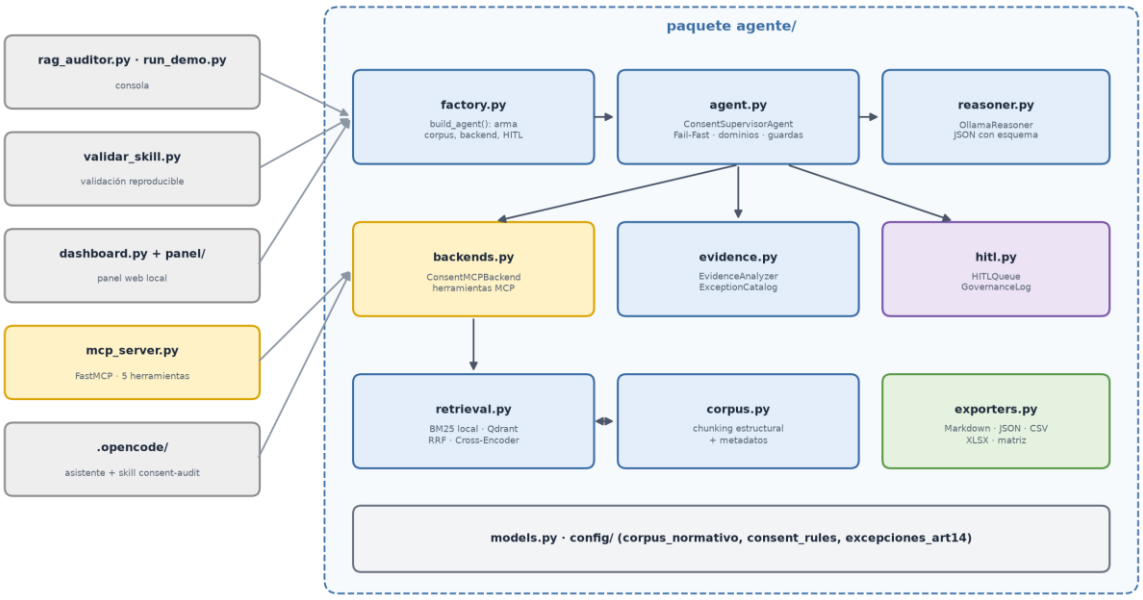


Figura 3: Diagrama de componentes

Componente	Responsabilidad	Elementos principales
`agent.py`	Orquesta la auditoría: Fail-Fast, evaluación por dominio, guardas, decisión HITL y registro estandarizado. Emite avisos de progreso opcionales.	`ConsentSupervisorAgent.audit_evidence()`
`backends.py`	Bus de herramientas MCP sobre el motor de recuperación elegido; registra cada llamada y protege la emisión.	`ConsentMCPBackend`, `build_backend()`
`retrieval.py`	Recuperación: BM25 en Python (modo local) o híbrida en Qdrant (densa + BM25 → RRF → Cross-Encoder), con filtros de metadatos.	`LocalBM25Retriever`, `QdrantHybridRetriever`
`corpus.py`	Chunking estructural por artículo, numeral y control, con metadatos trazables; ingesta de PDF.	`NormativeCorpus`, `fragment_document()`
`evidence.py`	Reglas deterministas por dominio, detección de negaciones, contradicciones e inyección; catálogo del Art. 14.	`EvidenceAnalyzer`, `ExceptionCatalog`
`reasoner.py`	Cliente de Ollama con salida JSON validada por esquema y temperatura 0.0.	`OllamaReasoner`
`hitl.py`	Cola persistente de revisión humana y bitácora de gobernanza.	`HITLQueue`, `GovernanceLog`
`exporters.py`	Informe Markdown, JSON, CSV, Excel y matriz de evidencias.	`render_markdown()`, `build_evidence_matrix()`
`factory.py` / `models.py`	Ensamblado del agente y constantes de dominio (controles, artículos, dictámenes).	`build_agent()`, `DOMAINS`
Puntos de entrada	Consola, panel web, servidor MCP, validación y asistente de OpenCode.	`rag_auditor.py`, `dashboard.py`, `mcp_server.py`, `validar_skill.py`, `.opencode/`

5. Vista de Ejecución

5.1. Secuencia de una auditoría

La Figura 4 muestra la interacción entre componentes al auditar una evidencia. El orden de los pasos es deliberado: las excepciones legales se evalúan **antes** que cualquier control de consentimiento (patrón Fail-Fast), y las guardas se aplican **después** del LLM y antes de la decisión.

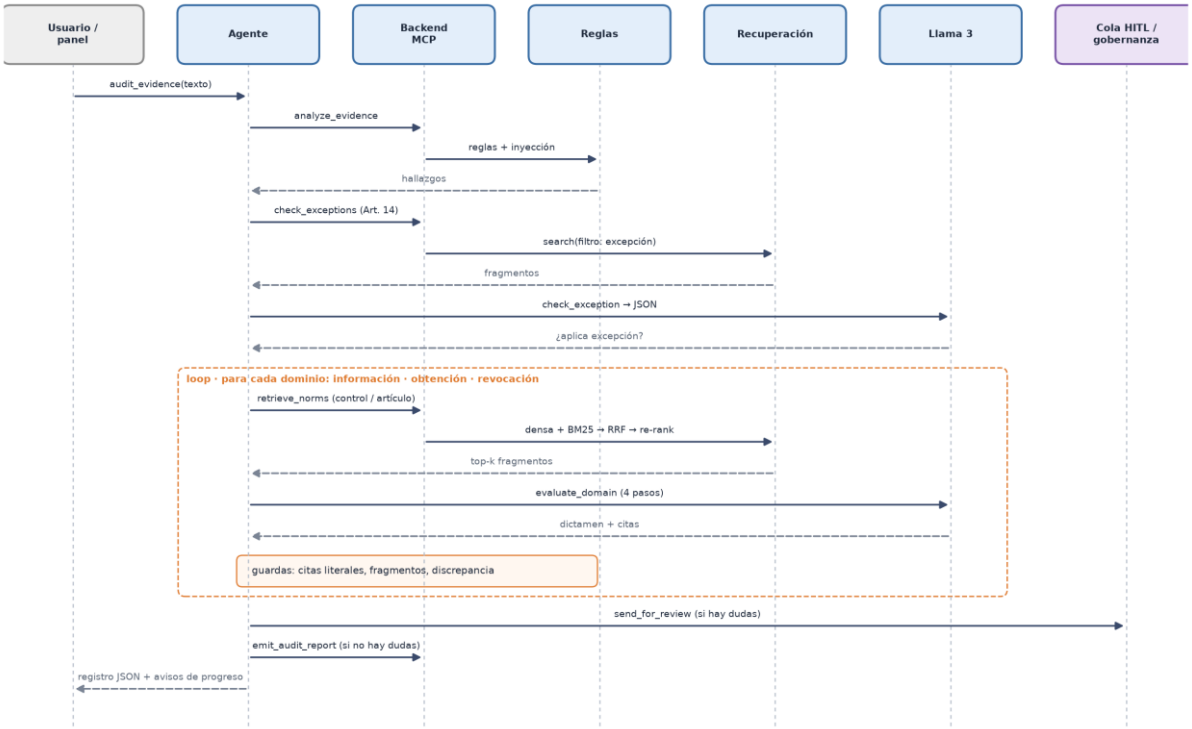


Figura 4: Diagrama de secuencia de una auditoría

1. **Entrada y análisis:** se valida que la evidencia no esté vacía, se calcula su huella SHA-256 y el motor de reglas evalúa cada requisito y busca instrucciones maliciosas.
2. **Fail-Fast:** se recuperan solo los fragmentos con `articulo = 14` y `naturaleza = excepcion`; reglas y LLM determinan si aplica una excepción. Una excepción solo se aplica si ambos coinciden.
3. **Recuperación por dominio:** dos búsquedas filtradas (controles ISO, incluidas las guías del Anexo B, y artículos de la ley) con hasta 4 fragmentos cada una.
4. **Razonamiento:** el LLM completa en JSON la exigencia, la cita, la brecha y el veredicto. Sin Ollama, se usa el dictamen determinista de las reglas.
5. **Guardas:** solo se aceptan citas literales y fragmentos recuperados; ante discrepancia se mantiene el dictamen de las reglas.
6. **Decisión y salida:** si existe algún motivo de escalamiento, el informe pasa a la cola del auditor con un ticket; si no, se emite a la bitácora. En ambos casos se devuelve el registro JSON.

5.2. Estados de un informe

Estado	Cuándo ocurre	Siguiente estado
`EMITIDO`	Ninguna condición de escalamiento se cumple.	Final (registrado en `governance_log.json`).
`PENDIENTE_REVISION_AUDITOR`	Excepción aplicada, falta de evidencia o de contexto, contradicción, inyección, discrepancia o citas no verificables.	`APROBADO` o `RECHAZADO` por el auditor.
`APROBADO`	El auditor confirma el informe con una nota.	Consolidado en `informes_finales.json` y la bitácora.

Estado	Cuándo ocurre	Siguiente estado
`RECHAZADO`	El auditor lo devuelve con una nota.	Final; queda la decisión en la matriz de evidencias.

6. Vista de Datos

6.1. Corpus normativo y metadatos

El corpus se fragmenta por unidad lógica: un artículo de la ley (el Art. 14, por numeral, con su cabecera antepuesta) o un control de la ISO (Anexo A como requisito y Anexo B como guía). Cada fragmento lleva los metadatos de la Tabla 3, que habilitan los filtros de recuperación y la trazabilidad del dictamen. El corpus base contiene 20 fragmentos resumidos por el equipo (la ISO está protegida por derechos de autor); los PDF oficiales con texto se incorporan automáticamente con texto literal.

Metadato	Uso en la arquitectura	Ejemplo
`norma`, `tipo_de_documento`	Filtrar por instrumento; citar la fuente	ISO/IEC 27701:2025 · ley nacional
`control`, `articulo`, `numeral`	Recuperación filtrada por dominio	A.1.2.5 · 14 · 5
`naturaleza`	Habilita el Fail-Fast (`excepcion`)	requisito_normativo · guia · excepcion
`seccion_jerarquica`	Trazabilidad en el informe	Anexo A > Tabla A.1 > A.1.2.5
`tipo_texto`, `origen`, `pagina`	Indicar si el texto es literal o resumen y su procedencia	resumen · corpus_normativo.json

Tabla 3: Metadatos de los fragmentos del corpus

6.2. Configuración externa

- `config/consent_rules.json`: reglas por dominio (tipo requerido o prohibido, patrones, patrones de contenido mínimo, controles, hallazgo, medida y prioridad) y patrones de inyección.
- `config/excepciones_art14.json`: supuestos del Art. 14 con sus patrones y los fines que nunca ampara (publicidad, marketing, perfilamiento).
- `config/corpus_normativo.json`: corpus base con los metadatos de la Tabla 3.

6.3. Registro de auditoría (salida estandarizada)

Cada auditoría produce un objeto JSON con la misma estructura que el agente de inventario del grupo, consumible por otros agentes. Sus bloques principales son:

```
{ "audit_id": "CON-2026-TIENDA-EB78B50004E3", "agent_id": "AGT-CONSENT-01",
  "subject": {"organization", "scope", "evidence_file", "evidence_sha256"},
  "engine": {"retriever": "qdrant", "llm": "llama3", "temperature": 0.0},
  "fail_fast": {"exception_applies", "justification", "matched", "fragments"},
  "domains": [{"domain", "controls", "articles", "dictamen", "evidencia_citada",
               "hallazgo", "medida_correctiva", "fragments", "rule_findings", "source"}],
  "overall_status": "NO_CONFORME", "corrective_actions": [...],
  "review_status": "PENDIENTE_REVISION_AUDITOR",
  "hitl_trigger": {"requires_auditor_signoff", "reason", "ticket"},
  "traceability": [{"domain", "fragment_id", "norma", "control", "articulo", "score"}] }
```

6.4. Archivos de salida

Archivo	Contenido
`informe_<caso>.md`	Informe legible con la tabla de dictámenes por dominio y la trazabilidad de fragmentos
`informes_draft.json / .csv / .xlsx`	Dictámenes normalizados, una fila por dominio
`matriz_evidencias.json`	Consultas al corpus, reglas aplicadas, Fail-Fast y decisiones del auditor
`pending_reviews.json`	Cola HITL persistente con ticket, registro y estado
`governance_log.json / informes_finales.json`	Bitácora de informes emitidos y consolidado de aprobados

7. Interfaces

7.1. Herramientas MCP

Herramienta	Entrada	Salida
`mcp_tool_retrieve_norms`	Consulta, top-k, filtros (norma, control, artículo, naturaleza)	Fragmentos con metadatos y puntaje
`mcp_tool_check_exceptions`	Texto de la evidencia	Aplica o no, numeral, justificación, fragmentos
`mcp_tool_analyze_evidence`	Texto de la evidencia	Estado de cada regla, citas y sospechas de inyección
`mcp_tool_emit_audit_report`	Registro generado por el flujo	Confirmación o rechazo (no acepta registros fabricados)
`mcp_tool_audit_consent`	Texto y nombre de la evidencia	Registro JSON e informe Markdown

7.2. Interfaces de usuario y de línea de comandos

Interfaz	Uso
`python dashboard.py`	Panel web local: proceso en vivo, resultado y cola del auditor (sección 8)
`python validar_skill.py`	Validación reproducible de los 5 casos, sin LLM ni decisiones simuladas
`python rag_auditor.py --evidencia <archivo>`	Auditoría de una evidencia por consola
`python run_demo.py [--modo qdrant]`	Demostración completa con todos los entregables
`python mcp_server.py`	Servidor MCP (stdio) para OpenCode u otros agentes
OpenCode: asistente `consent-supervisor` + skill `consent-audit`	Auditoría conversacional; el skill se activa automáticamente ante solicitudes de consentimiento

7.3. API del panel web

Ruta	Método	Función
`/`	GET	Página del panel
`/api/estado`	GET	Motor, IA disponible, tamaño del corpus y pendientes
`/api/casos`	GET	Evidencias disponibles en `evidencias/`
`/api/auditar`	POST	Ejecuta la auditoría y transmite un evento por paso (NDJSON) y el resultado final
`/api/pendientes`	GET	Cola del auditor
`/api/decidir`	POST	Aprueba o rechaza un ticket con nota y consolida los aprobados
`/api/informe?id=`	GET	Descarga del informe en Markdown

8. Panel Web de Demostración

Para mostrar el funcionamiento sin recurrir a la terminal se incorporó un panel web local (`dashboard.py` y `panel/index.html`). Se implementó con la biblioteca estándar de Python y sin librerías ni fuentes externas, por lo que funciona sin conexión a Internet y solo escucha en `127.0.0.1`. El agente expone avisos de progreso opcionales (`on_event`) que no alteran su resultado; el servidor los transmite al navegador en tiempo real, y la página enciende cada paso a medida que el agente trabaja (Figura 5). Al terminar se muestran los dictámenes por dominio con sus citas y normas, las medidas correctivas, los motivos de revisión y la cola del auditor con los botones de aprobar y rechazar (Figura 6).

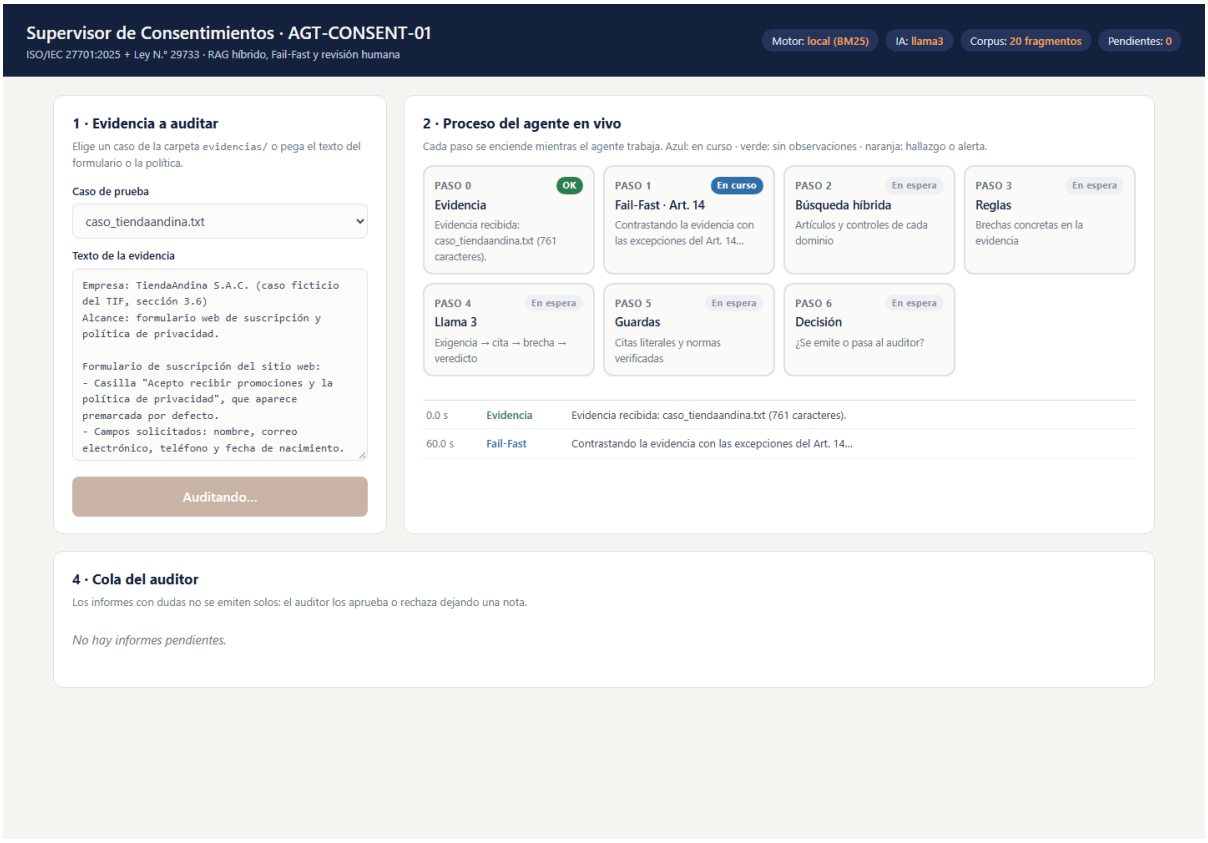


Figura 5: Panel web durante la auditoría (Fail-Fast en curso, con Llama 3)



Figura 6: Resultado de la auditoría del caso TiendaAndina en el panel

9. Vista de Despliegue

Todos los componentes se ejecutan en el equipo del auditor (Figura 7). El proceso Python aloja el núcleo, el panel y, en modo híbrido, Qdrant embebido en la carpeta `.qdrant_local/` y los modelos de FastEmbed en `.modelos/`, por lo que **no se requiere Docker**. Ollama corre como servicio local con Llama 3. Solo el asistente de OpenCode, opcional, utiliza un modelo en línea.

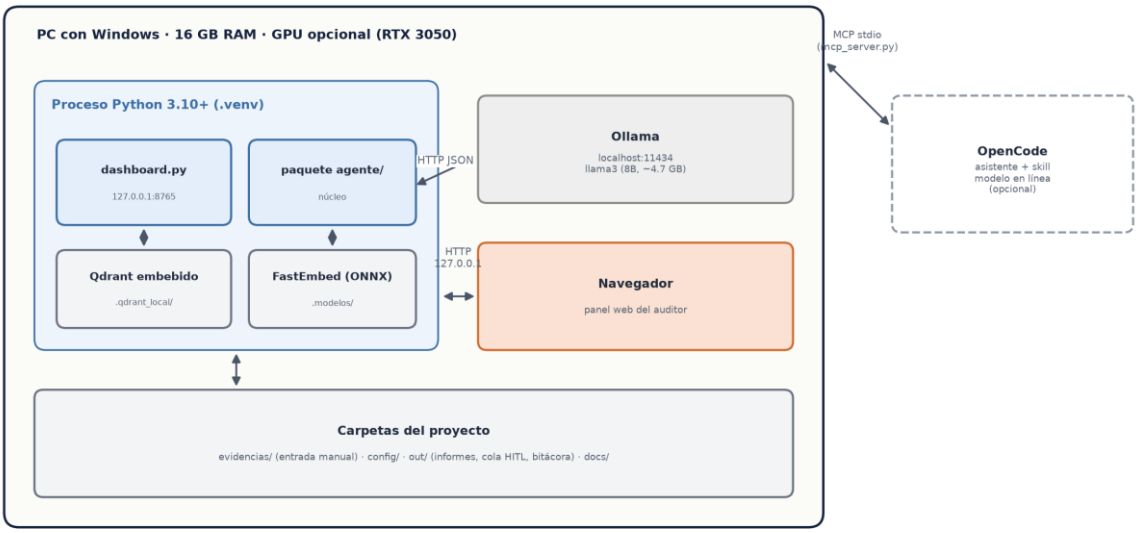


Figura 7: Diagrama de despliegue en el equipo del auditor

Recurso	Requisito	Observación
Sistema y Python	Windows, Python 3.10+ y entorno `.venv`	`pip install -r requirements.txt`
Memoria	16 GB de RAM recomendados	Llama 3 (8B) ocupa unos 4.7 GB
GPU	Opcional (probado con NVIDIA RTX 3050)	Acelera el LLM; sin GPU funciona más lento
Ollama	Servicio local en `localhost:11434` con `llama3`	Sin Ollama, el agente usa el dictamen de las reglas
Modelos de búsqueda	~800 MB la primera vez en modo qdrant	Multilingüe MiniLM, BM25 y bge-reranker-v2-m3
Tiempos medidos	Modo local: segundos · con Llama 3: ~2 min por caso	Fail-Fast ~20 s y ~40 s por dominio

10. Seguridad, Control y Trazabilidad

- **Revisión humana (HITL):** el informe se retiene si se aplicó una excepción, falta evidencia o contexto normativo, hay contradicciones, se detecta inyección de instrucciones o el LLM discrepa o no aporta citas literales.
- **Guardas anti-alucinación:** citas literales y contiguas (conservan negaciones), fragmentos efectivamente recuperados y prevalencia de las reglas ante discrepancias.
- **Emisión protegida:** `mcp_tool_emit_audit_report` solo acepta registros íntegros generados por el propio flujo; rechaza pendientes o fabricados.
- **Integridad y unicidad:** huella SHA-256 de la evidencia, identificador único por auditoría y ticket persistido desde su creación.
- **Robustez ante manipulación:** la evidencia se trata como dato en el prompt y las instrucciones incrustadas se detectan y escalan.
- **Procesamiento local:** LLM, índice y panel locales; el panel solo escucha en `127.0.0.1` y limita el tamaño de la evidencia.

- **Auditoría de caja blanca:** reglas, excepciones y corpus en JSON versionable; cada llamada a herramientas queda en la matriz de evidencias.

11. Calidad y Validación

La arquitectura se valida con **45 pruebas automatizadas** (pytest), todas aprobadas, organizadas en tres archivos: comportamiento funcional del agente (`test_agent.py`), regresiones de seguridad del segundo entregable (`test_safety_regressions.py`) y el panel con sus avisos de progreso (`test_dashboard.py`). Además, `validar_skill.py` comprueba de forma reproducible que los 5 casos de prueba (TiendaAndina, Banco Andino, LibreríaSur, ClubFit y formulario incompleto) producen el estado esperado, y la demostración completa con Llama 3 obtuvo los mismos 15 dictámenes de dominio.

12. Decisiones de Arquitectura

Decisión	Alternativa descartada	Motivo
Patrón Fail-Fast sobre el Art. 14	Evaluar todos los controles siempre	Evita no conformidades falsas cuando la ley no exige consentimiento
RAG híbrido (densa + BM25 → RRF → Cross-Encoder)	Solo búsqueda densa	Combina significado y coincidencia exacta de artículos y códigos de control
Chunking estructural por artículo y control	Cortes de longitud fija	Una excepción nunca queda separada de la regla que modifica
LLM local (Llama 3 en Ollama, T = 0)	Modelo en la nube	La evidencia no sale del equipo; resultados deterministas
Reglas verificables como dictamen provisional	Confiar en el LLM	Llama 3 acertó 8/10 en la evaluación; el LLM es una segunda opinión
Bus de herramientas MCP	Llamadas directas entre módulos	Contratos explícitos, trazabilidad y uso por otros agentes
Qdrant embebido	Servidor en Docker	Despliegue simple en cualquier laptop del equipo
Panel con biblioteca estándar	Framework web externo	Sin dependencias ni Internet; misma lógica que la consola

13. Limitaciones y Evolución

- El corpus base contiene resúmenes de la ISO; el texto literal requiere los PDF oficiales con texto (el disponible es escaneado y necesita OCR).
- Las reglas y el catálogo de excepciones se basan en patrones y no constituyen un verificador jurídico completo; redacciones no previstas requieren revisión humana.
- Las reglas conservadoras derivan al auditor la mayoría de los casos, lo que aumenta la carga de revisión.
- Pendiente técnico: persistencia concurrente de la cola, protección de fórmulas en Excel y versionado completo de decisiones.
- **Tercera unidad:** orquestación con AGT-DATA-INV-01 (auditar automáticamente las actividades del RoPA con base legal «Consentimiento»), validación con evidencias reales y evaluación de modelos de mayor tamaño.